KERN

Infrastructure for distributed energy.

Grid-Serving Storage

TEB AI Hackathon — Submission

Critical Infrastructure & Energy Security

Riddle&Code GmbH — Vienna

Jürgen Eckel · Thomas Fürstner

October 2026

TEB AI HACKATHON

CRITICAL INFRASTRUCTURE & ENERGY SECURITY

Submission

1. Project basics

Project title

Grid-Serving Storage

One-line pitch

An interoperability profile and live demo that lets a Technical Aggregator coordinate distributed battery storage across heterogeneous HEMS backends via MQTT and OpenADR to relieve grid congestion — without overriding prosumer autonomy.

2. Team

Team name

RIDDLE&CODE

Team members

Jürgen Eckel — RIDDLE&CODE GmbH — @eckelj — CEO, profile design & demo

Thomas Fürstner — RIDDLE&CODE GmbH — @annonymmous — CTO, OpenADR 3.1 VEN reference implementation

Primary contact email

juergen@riddleandcode.com — +43 660 794 3291

3. Challenge track

Track

Grid-Friendly / Grid-Serving Storage

Challenge prompt (if your track had more than one)

“Grid-Serving Storage” — using distributed customer storage capacity (residential through commercial) as a fast-response measure against grid overloads in distribution networks.

4. What you built

Problem you solved

Distribution grids face growing stress from renewable build-out (PV, wind) and electrification loads (heat pumps, EV chargers), while grid expansion is slow and expensive. Meanwhile, distributed battery storage is being deployed at scale — primarily for prosumer self-consumption. This storage capacity could relieve grid bottlenecks, but there is no standardized, reusable interface for a DSO or aggregator to coordinate thousands of heterogeneous storage systems across different HEMS manufacturers. Without such an interface, the flexibility sits idle while curtailment increases.

Solution overview

We designed a rough interoperability profile following the IES methodology and the Austrian system architecture for flexible assets (Smart Grids Austria / AIT). The profile places the Technical Aggregator as the central coordination role — the DSO signals grid stress to the aggregator, who dispatches flexibility goals to HEMS Backends via MQTT v5. Each backend translates these goals into per-device setpoints via OpenADR (2.0b or 3.1) to HEMS devices, which evaluate locally and opt in or out. The system operates as a continuous four-phase loop (Observe → Plan → Dispatch → Settle): backends continuously publish state snapshots and flexibility offers, the aggregator runs a portfolio-level MILP to plan optimal dispatch, sends goals top-down when triggered, and collects verified delivery reports for settlement. A two-level hierarchical MILP — aggregator across backends, backend across devices — ensures privacy (the aggregator never sees per-device data), scalability (solving for backends, not thousands of devices), and manufacturer autonomy. A live demo with three simulated HEMS backends, a real MQTT broker, real MILP optimization (PuLP + HiGHS), and an interactive browser dashboard demonstrates the full cycle end-to-end.

How AI is used

AI is used at two levels as Mixed-Integer Linear Programming (MILP) optimization. The aggregator MILP treats each HEMS Backend as an opaque flexibility envelope and determines how much to request from each backend to meet a grid obligation — minimizing portfolio peak power while penalizing unreliable backends via reliability scores. Each backend then runs its own device-level MILP to distribute the received goal across individual devices, respecting per-device SoC, battery limits, ramp rates, and prosumer preferences. Both solvers use PuLP with HiGHS — real mixed-integer linear programming, not heuristics or random allocation. The human stays in the loop at the device level: every HEMS device evaluates the request against local constraints and the prosumer's configured preferences, and can opt out.

Security and resilience

The primary threat model is fail-safe degradation: any communication failure (MQTT broker down, backend offline, device unreachable) defaults to normal prosumer-controlled operation — no aggregator command can persist without active renewal. MQTT v5 message expiry prevents stale flex-requests from activating devices after a reconnect. The HEMS device is the trust boundary — it enforces local constraints (SoC floor, hardware limits, user preferences) regardless of what the aggregator or backend requests. TLS 1.3 secures all MQTT connections with mutual client certificate authentication; OpenADR uses PKI (2.0b) or OAuth 2.0 (3.1). ACLs restrict each backend to publishing only on its own topics. The profile does not model cyber-attacks on the broker or protocol-level exploits — it focuses on operational resilience through soft signaling and local device autonomy.

Data and environments

All data is synthetic. The demo simulates three HEMS backend portfolios (47 residential, 12 commercial, 25 mixed devices) with randomized but realistic device state (SoC 25–90%, consumption patterns, PV output). Flexibility envelopes are computed from simulated de-

vice state, not learned from historical data. The MILP solvers (PuLP + HiGHS) are real. MQTT communication runs through a real Mosquitto broker. OpenADR device interactions are simulated in-process (logged, not actual HTTP/XML). Spot price data for the OpenADR 3.1 VEN reference implementation uses the aWATTar Austria API (public, free). No live infrastructure, venue network, or third-party systems were tested.

Tech stack

- **Backend**: Python 3.12+
- **Optimization**: PuLP + HiGHS (MILP solver)
- **Messaging**: MQTT v5 via Mosquitto broker, paho-mqtt client library
- **Device protocol**: OpenADR 2.0b/3.1 (simulated in demo; reference VEN implementation in Python with openleadr-rs VTN)
- **Frontend**: HTML/JS dashboard with Chart.js, MQTT.js (websocket subscription)
- **Containerization**: Docker Compose (Mosquitto + 3 backends + aggregator)
- **Reference data**: aWATTar Austria API (spot prices for OpenADR 3.1 demo)

5. Links

Code repository

https://github.com/RiddleAndCode/grid-serving-storage

Demo link

Run locally via `docker compose up --build` then open `demo/dashboard.html` — or single-process: `cd demo && python3 run_demo.py` for a 50-second terminal demo of the full Observe → Plan → Dispatch → Settle cycle.

Documentation

https://github.com/RiddleAndCode/grid-serving-storage/blob/main/README.md

The profile documentation is in `profile/01-use-case.md` through `profile/07-interface-comparison-at-de.md`. The aggregator interface spec (TypeScript type definitions + MILP formulation) is in `aggregator-interface-spe`

Pitch deck (optional)

https://github.com/RiddleAndCode/grid-serving-storage/blob/main/pitch-deck.md

Other links (optional)

- Protocol analysis (MQTT vs. OpenADR fitness across markets): `protocol-analysis-mqtt-vs-openadr.md`
- Austrian vs. German interface comparison: `profile/07-interface-comparison-at-de.md`
- Smart Grids Austria system architecture poster: `challenge/poster-systemarchitektur-v2g.md`

6. Notes for the jury

- The **interoperability profile** is the core deliverable — the demo exists to make the profile tangible, not the other way around. The profile covers use case, actors, process flow, information model, communication layer, and design decisions across 7 documents fol-

lowing the IES methodology.

- The OpenADR device interface is **simulated** in the demo (logged in-process, not real HTTP/XML exchanges). A separate reference VEN implementation (OpenADR 3.1) validates the mapping against a real VTN but is not integrated into the demo pipeline.
- The MILP optimization is **real** — both the aggregator-level and backend-level solvers use PuLP with HiGHS. The two-level hierarchical pattern (aggregator optimizes across backends, backend optimizes across devices) is a key architectural contribution.
- Flexibility envelopes in the demo are **computed from simulated device state**, not learned from historical ramp behavior as described in the profile. In production, the flex_up/flex_down values would be derived from observed device behavior over time.
- The profile supports **both OpenADR 2.0b and 3.1** on the device interface. The HEMS Backend abstracts the version difference — the aggregator-facing MQTT interface is identical regardless of which OpenADR version the devices speak.
- The profile is deliberately **Austrian-model-specific** (Technical Aggregator, no Smart Meter Gateway dependency, OpenADR as preferred device protocol) but the interface design is reusable across DSOs and device manufacturers.

7. Follow-up (optional)

[X] We'd like an invitation to TEB Klub, TEB's community platform.

[X] TEB and the challenge partners may contact our primary contact about our project.

[X] TEB may share our project title, description and event photos on its channels.